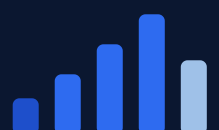


Assemble, Don't Build.

Composing your own agent layer from the parts that are already commodity — and the [two parts still worth building yourself](#).

The harness matters more than the model. By 2026 most of the harness is a solved, off-the-shelf problem. The only question left is *which layer you build* — and why the answer is almost always “fewer than you think.”



STOP ASKING BUILD-VS-BUY

It was never build-vs-buy. It's **which layer**.

The era of monolithic “agent frameworks” is closing. The pieces that used to arrive as one sealed product now ship as narrow, composable parts: a coding harness here, a terminal multiplexer there, a model router beside it. Each does one job and gets out of the way.

Reusing those parts isn't cheating, and it isn't dodging a subscription meter — it's **composition**. You orchestrate sanctioned, well-scoped components and own only the layer above them. The mistake isn't buying. The mistake is re-building a solved problem because building feels like progress. (If you have a named weakness for infrastructure-before-shipping, this is exactly where it bites.)

So the useful question isn't “should I build my own?” It's asked once per layer, and it has two parts.

THE TWO-QUESTION TEST

Run it on every layer of your stack.

1 · Is it commodity?

Commodity + no judgment → reuse. Many good, fast-converging open options exist; whatever you'd build is a worse version of what you can install.

2 · Does it encode a judgment only I can make?

Encodes your judgment → own it. No product ships your decisions.

Everything in between is you **rebuilding a solved problem**.

APPLY THE TEST TO A REAL STACK

Six layers. Four you rent. Two **you own**.

A distributed agent layer — the kind you'd build to run jobs across your own machines — decomposes into six concerns. Only two of them survive the two-question test as “build.”

1

Executor — the agent loop

The thing that actually reads, edits, and runs. Reuse: a minimal harness (four-tool core, 20+ providers incl. local models, embeddable via print-JSON / RPC / SDK) or a Rust harness built for many parallel sessions (10 MB each).

Rent

2

Multiplexer + agent state

Keep sessions alive; know who's blocked, working, done. Reuse: a tmux rebuilt for agents, exposing a socket API that emits state instead of making you poll terminal output with regex.

Rent

3

Model routing

Cheap worker for the bulk, a strong model for planning and review; local + cloud. Reuse: a router, or the harness's own unified provider layer. Never hand-roll provider switching.

Rent

4

Reach — notify & drive from a phone

See a job finish; steer it from away. Reuse: the multiplexer's remote mode over SSH plus a chat bridge. Fully off-the-shelf.

Rent

5

Dispatch topology

How a job travels from your cockpit to the executor and the result back — across your machines, your sync, your always-on box. This encodes how you work. No product ships your topology.

Build

6

Gate policy

What runs freely versus what stops for a human. Pure judgment — and the most popular minimal harness ships zero permission controls, on purpose. The gate is never a feature you install.

Build

This shape isn't exotic. A public reference architecture already swaps a plain multiplexer for an agent-aware one and turns its state-events into an inbox — no polling, jobs delivered as they change. Proof the pattern is real infrastructure, not a lab toy. **The parts are commodity; the arrangement is yours.**

LAYER 6, UP CLOSE

A human in the loop isn't a strategy. It's a cost.

Gate everything and you get **approval fatigue**: you rubber-stamp what you stopped reading — the same failure mode as security-alert fatigue. A wall of confirmations isn't safety. It's safety theatre that trains you to click "yes."

So gate **strategically**. Hold the human on the actions that are **irreversible, high-blast-radius, judgment-heavy, or not-yet-trusted**. Let the reversible, contained, verifiable, repetitive ones run — and report. Subagents make this scale: autonomy on the leaves, one gate at the trunk.

THE FOUR-QUESTION GATE TEST

Before any action runs unattended, ask:

Reversible?**Contained?****Verifiable?****Trusted?**

Four yeses → let it run and report. Any single no → stop for a human. The test is cheap to run and it's the whole policy — you're not writing rules, you're classifying blast radius.

One invariant survives full autonomy: **observability**. The danger was never autonomy — it's *silent* autonomy. An agent that acts without leaving a legible trail is the actual risk. So the one rule you never relax, no matter how much you let run: **fail loud**.

THE COST OF RENTING

The moment you assemble, the trust boundary becomes yours.

Every rented part is code you didn't write and a marketplace you don't vet. These minimal harnesses will happily print a secret to the screen the instant they read your config, and most extension marketplaces are unreviewed by default. Reuse aggressively — but the seams between the parts, the credentials they hold, and the data they can see are now **your job**, not the vendor's. Right-size it: a stated boundary matched to the real risk, not a fortress you'll never finish.

MAP YOUR OWN STACK

RENT / BUILD · ONE REASON

1 • Executor

the agent loop

☐

Rent

☐

Build

2 • Multiplexer + state

alive & observable

☐

Rent

☐

Build

3 • Model routing

cheap worker → strong reviewer

☐

Rent

☐

Build

4 • Reach

notify & drive from away

☐

Rent

☐

Build

5 • Dispatch topology

how you work across machines

☐

Rent

☐

Build

6 • Gate policy

run freely vs. stop for a human

☐

Rent

☐

Build

The one-line rule of thumb. If a good open option exists and the layer doesn't encode a decision only you can make — rent it, and spend the hours you saved on the two layers that are actually yours: how the work moves, and what you let it do alone.

